Архитектура приложений на C#

Введение

Настоящая лекция посвящена трём концепциям, составляющим основу архитектуры большинства современных .NET-приложений:

- **Слоистая архитектура** — способ организации кода по областям ответственности
- **Domain-Driven Design** — подход к моделированию бизнес-логики
- **Dependency Injection** — механизм управления зависимостями между компонентами

Все три концепции независимы от конкретного фреймворка и применимы в любом проекте.

Симптомы неструктурированного кода

Рассмотрим класс `OrderService`, который разрастался по мере добавления требований: создание заказа, отмена, обновление статуса, расчёт скидок, поиск, экспорт, интеграция с логистикой.

```
public class OrderService
{
    // Создание: валидация, расчёт скидки, сохранение
    public async Task CreateOrderAsync(...) { ... }

    // Отмена, частично дублирует валидацию из Create
    public async Task CancelOrderAsync(...) { ... }

    // Обновление статуса
    public async Task UpdateStatusAsync(...) { ... }

    // Расчёт итоговой суммы, логика скидок повторяется
    public decimal CalculateTotal(...) { ... }

    // Поиск SQL-запросов непосредственно в методе
    public async Task<List<Order>> SearchAsync(...) { ... }

    // Экспорт, содержит знание о формате CSV
    public async Task ExportToCsvAsync(...) { ... }

    // Интеграция с логистикой HTTP-запросов
    public async Task SyncWithLogisticsAsync(...) { ... }
}
```

Данный класс обладает знаниями обо всём: о структуре базы данных, о формате CSV, о внешнем API логистики, о бизнес-правилах скидок, о допустимых переходах статусов. Подобная конструкция носит название **«God Class»** — класс, взявший на себя ответственность за всё.

На практике это проявляется в четырёх характерных симптомах.

Симптом 1: изменение в одном месте нарушает работу другого. Правка логики расчёта скидки требует поиска всех мест её дублирования: в `CreateOrderAsync`, в `CalculateTotal`, в `CancelOrderAsync`. Пропущенное вхождение приводит к тому, что в рабочей среде один и тот же заказ рассчитывается по-разному в зависимости от пути выполнения.

Симптом 2: модульное тестирование требует полной инфраструктуры. Проверка корректности расчёта скидки для заказа на определённую сумму невозможна в изоляции, поскольку `CreateOrderAsync` требует реального `AppDbContext` и реального HTTP-клиента. Для тестирования одного бизнес-правила необходима вся инфраструктура.

Симптом 3: замена технологии сопряжена с риском. Например смена СУБД осложняется тем, что строка подключения задана непосредственно в коде в нескольких местах, а часть запросов написана с синтаксисом, специфичным для текущей СУБД.

Симптом 4: ориентирование в коде затруднено. Вопрос «где реализована логика скидок» требует прочтения всего `OrderService`. Вопрос «где сохраняется заказ» — изучения трёх методов, каждый из которых создаёт `AppDbContext` по-своему.

Все четыре симптома являются следствием **высокой связанности**: единый класс содержит знания о слишком большом числе не связанных между собой вещей. Изменение в любой из них потенциально затрагивает всё остальное.

1. Слоистая архитектура

Слоистая архитектура — наиболее распространённый способ структурирования кода в .NET-приложениях. Основная идея состоит в разделении приложения на группы компонентов, каждая из которых отвечает за строго определённую область, при установленных правилах взаимодействия между ними.

Четыре слоя

Типовая слоистая архитектура в .NET состоит из четырёх слоёв.

Domain — основа приложения. Содержит сущности, бизнес-правила и инварианты. Это единственный слой, независимый ни от каких других: ни от базы данных, ни от фреймворков, ни от пользовательского интерфейса.

```
Domain/
├── Entities/
│   └── Order.cs              — сущность с поведением
├── ValueObjects/
│   └── Money.cs              — денежная сумма с валидацией
├── Repositories/
│   └── IOrderRepository.cs   — интерфейс, реализация находится в
Infrastructure
└── Exceptions/
    └── DomainException.cs    — исключения предметной области
```

Ключевой признак корректности Domain-слоя: в проекте должна быть только стандартная библиотека .NET и ссылки на другие классы того же слоя.

Application — сценарии использования. Координирует взаимодействие между Domain и Infrastructure для выполнения конкретных операций: создать заказ, отменить заказ, получить список. Слой не содержит бизнес-логики — только оркестрацию (согласование действий). Стандартный способ организации этого слоя — паттерн **Command + Handler**: каждый сценарий использования представлен отдельной парой классов, где Command описывает намерение, а Handler знает как его выполнить.

```
Application/
└── Orders/
    ├── CreateOrderCommand.cs
    ├── CreateOrderHandler.cs
    └── GetOrdersHandler.cs
```

Infrastructure — технические детали. Реализует то, что Domain объявляет через интерфейсы. Здесь располагается весь код, связанный с конкретными технологиями: Entity Framework Core, HTTP-клиенты, файловая система.

```
Infrastructure/
├── Persistence/
│   ├── AppDbContext.cs
│   └── EfOrderRepository.cs     — реализует IOrderRepository
└── Api/
    └── HttpService.cs      — реализует IHttpService
```

Presentation — взаимодействие с пользователем. В WPF-приложении это Views и ViewModels. Слой не содержит бизнес-логики и не обращается к базе данных напрямую: он отображает данные и транслирует действия пользователя в вызовы Application-слоя.

```
Presentation/
├── Views/
│   └── OrdersView.xaml
├── ViewModels/
│   └── OrdersViewModel.cs
└── App.xaml.cs               — точка входа и конфигурация зависимостей
```

Существует основополагающее правило слоистой архитектуры, нарушение которого сводит на нет все её преимущества — **зависимости направлены исключительно к центру, к Domain**. Presentation знает об Application. Application знает о Domain. Infrastructure знает о Domain — реализует объявленные им интерфейсы. Domain не зависит ни от кого.

Конкретные примеры нарушений данного правила:

```
// Нарушение 1: Domain содержит атрибут из Infrastructure
public class Order
{
    [Column("order_id")]       // ❌ атрибут из пространства имён EF Core
    public Guid Id { get; set; }
}

// Нарушение 2: Presentation обращается к DbContext напрямую
public class OrdersViewModel
{
    private readonly AppDbContext _context; // ❌ Infrastructure в
Presentation

    public async Task LoadAsync()
    {
```

```
        Orders = await _context.Orders.ToListAsync(); // ❌ Application и
Domain обходятся
    }
}

// Нарушение 3: Application зависит от конкретной реализации
public class CreateOrderHandler
{
    private readonly HttpClient _http; // ❌ конкретная реализация из
Infrastructure
}
```

Каждое из этих нарушений создаёт жёсткую связь: замена технологии становится невозможна без затрагивания логики по всему проекту.

Взаимодействие слоёв через интерфейсы

Интерфейсы — механизм, обеспечивающий соблюдение направления зависимостей. Domain объявляет, что ему необходимо; Infrastructure предоставляет конкретную реализацию.

```
// Domain/Repositories/IOrderRepository.cs
public interface IOrderRepository
{
    Task<Order?>      GetByIdAsync(Guid id);
    Task<List<Order>> GetAllAsync();
    Task<List<Order>> GetPendingAsync();
    Task              AddAsync(Order order);
    Task              UpdateAsync(Order order);
}
```

```
// Infrastructure/Persistence/EfOrderRepository.cs
public class EfOrderRepository : IOrderRepository
{
    private readonly AppDbContext _context;

    public EfOrderRepository(AppDbContext context)
        => _context = context;

    public async Task<Order?> GetByIdAsync(Guid id)
        => await _context.Orders.FirstOrDefaultAsync(o => o.Id == id);

    public async Task<List<Order>> GetAllAsync()
        => await _context.Orders
            .OrderByDescending(o => o.CreatedAt)
            .ToListAsync();

    public Task<List<Order>> GetPendingAsync()
```

```
        => _context.Orders
            .Where(o => o.Status == "Submitted")
            .OrderBy(o => o.CreatedAt)
            .ToListAsync();

    public async Task AddAsync(Order order)
    {
        _context.Orders.Add(order);
        await _context.SaveChangesAsync();
    }

    public async Task UpdateAsync(Order order)
    {
        _context.Orders.Update(order);
        await _context.SaveChangesAsync();
    }
}
```

Application вызывает `IOrderRepository` и не знает, что за этим интерфейсом стоит Entity Framework Core, SQLite или хранилище в оперативной памяти. Данное решение принимается в одном месте при сборке приложения — этому посвящён раздел Dependency Injection.

Критерии применимости

Слоистая архитектура — инструмент, а не обязательный стандарт. Разделение на четыре полноценных слоя оправдано при нетривиальной бизнес-логике, нескольких разработчиках или необходимости долгосрочного развития продукта.

Для небольших проектов с простой логикой достаточно упрощённой структуры: папки `Models`, `Services`, `Views` в рамках единого проекта с соблюдением тех же принципов направления зависимостей. Ключевое правило неизменно вне зависимости от масштаба: **каждый класс обладает ровно тем объёмом знаний, который необходим для выполнения его задачи**.

2. Domain-Driven Design

Domain-Driven Design (DDD) — это подход к проектированию, при котором структура кода отражает структуру предметной области. Центральная идея может быть сформулирована следующим образом: **код должен говорить на языке предметной области**.

Это означает, что разработчик в первую очередь думает не о таблицах, HTTP-запросах или событиях — а о заказах, клиентах, платёжных инструкциях, правилах списания. Технические детали являются средством реализации, но не языком проектирования.

Ubiquitous Language — единый язык предметной области

Один из фундаментальных принципов DDD — единый язык (*Ubiquitous Language*): термины предметной области используются одинаково в коде, документации и деловом общении с заказчиком. Отсутствие единого языка порождает постоянные потери на «перевод» между тем, что говорит бизнес, и тем, что написано в коде.

Практическое проявление этого принципа хорошо заметно в именовании:

```
// Технический язык — значение понятно только разработчику
order.SetStatus("Submitted");
repository.GetByStatus("Submitted");

// Язык предметной области — понятен всем участникам проекта
order.Submit();
repository.GetPendingAsync();
```

Исключения предметной области

Нарушения бизнес-правил принципиально отличаются от технических сбоев: «нельзя оформить пустой заказ» — это ожидаемая ситуация в предметной области, а «упала база данных» — техническая авария. Стандартный `ArgumentException` не позволяет их различить. Для этого в Domain объявляется собственный тип исключения:

```
// Domain/Exceptions/DomainException.cs
public class DomainException : Exception
{
    public DomainException(string message) : base(message) { }
}
```

`DomainException` всегда содержит понятное сообщение на языке пользователя: «Нельзя оформить пустой заказ», «Заказ уже отменён». Слой Presentation обрабатывает его особым образом, отличая от технических сбоев:

```
private async Task SubmitAsync()
{
    try
    {
        await _handler.HandleAsync(command);
    }
    catch (DomainException ex)
    {
        // Понятное сообщение из предметной области — отображается
пользователю
        ErrorMessage = ex.Message;
    }
    catch (Exception)
    {
        // Технический сбой — отображается общее сообщение
        ErrorMessage = "Произошла техническая ошибка. Попробуйте позже.";
    }
}
```

Технические исключения — `SqlException`, `HttpRequestException` — не передаются пользователю.

Анемичная модель

Прежде чем рассмотреть рекомендуемый подход, необходимо охарактеризовать антипаттерн, который встречается настолько часто, что нередко воспринимается как норма.

Анемичная модель — классы-контейнеры данных без поведения: набор свойств с геттерами и сеттерами, где вся логика вынесена во внешние сервисы.

```
// Анемичная модель: класс — контейнер данных без поведения
public class Order
{
    public Guid     Id           { get; set; }
    public string   CustomerName { get; set; } = "";
    public string   Status       { get; set; } = "New";
    public decimal  TotalAmount  { get; set; }
    public decimal  Discount     { get; set; }
    public List<OrderItem> Items { get; set; } = new();
}
```

```
// Логика вынесена в сервис, который вынужден знать о внутренней структуре
Order
public class OrderService
{
    public void SubmitOrder(Order order)
```

```
    {
        if (order.Items.Count == 0)
            throw new Exception("Нельзя оформить пустой заказ");

        if (order.Status != "New")
            throw new Exception("Заказ уже оформлен");

        // Прямое изменение состояния снаружи объекта
        order.Status = "Submitted";
    }

    public void AddItem(Order order, string productName, int quantity,
decimal price)
    {
        if (order.Status != "New")
            throw new Exception("Нельзя добавить товар в обработанный
заказ");

        if (order.Items.Count >= 50)
            throw new Exception("Нельзя добавить более 50 позиций");

        order.Items.Add(new OrderItem(productName, quantity, price));

        // Пересчёт суммы производится снаружи объекта
        var subtotal      = order.Items.Sum(i => i.Price * i.Quantity);
        order.Discount    = subtotal > 10_000m ? subtotal * 0.10m : 0m;
        order.TotalAmount = subtotal - order.Discount;
    }
}
```

Проблема в том, что логика расположена **снаружи объекта**. Ничто не препятствует другому разработчику написать неправильные данные напрямую, обойдя все проверки, или добавить позицию в список без пересчёта суммы. Объект не защищает корректность своего состояния.

Модель с поведением

Решение состоит в переносе логики внутрь объекта. **Инвариант** — условие, которое должно выполняться для объекта всегда, при любом состоянии. Если объект несёт ответственность за собственные инварианты, некорректное состояние становится физически недостижимым.

```
public class Order
{
    private readonly List<OrderItem> _items = new();

    public Guid                     Id           { get; private set; }
    public string                   CustomerName { get; private set; } = "";
```

```
    public IReadOnlyList<OrderItem> Items        => _items.AsReadOnly();
    public decimal                  TotalAmount  { get; private set; }
    public decimal                  Discount     { get; private set; }
    public string                   Status       { get; private set; } =
"New";

    // Приватный конструктор — создание объекта возможно только через
фабричный метод
    private Order() { }

    public static Order Create(string customerName)
    {
        if (string.IsNullOrWhiteSpace(customerName))
            throw new DomainException("Имя клиента обязательно");

        return new Order
        {
            Id           = Guid.NewGuid(),
            CustomerName = customerName.Trim()
        };
    }

    public void AddItem(string productName, int quantity, decimal price)
    {
        if (Status != "New")
            throw new DomainException("Нельзя добавить товар в обработанный
заказ");

        if (_items.Count >= 50)
            throw new DomainException("Нельзя добавить более 50 позиций");

        _items.Add(new OrderItem(productName, quantity, price));
        RecalculateTotal();
    }

    public void Submit()
    {
        if (_items.Count == 0)
            throw new DomainException("Нельзя оформить пустой заказ");

        if (Status != "New")
            throw new DomainException("Заказ уже оформлен");

        Status = "Submitted";
    }

    public void Cancel()
    {
        if (Status == "Delivered")
            throw new DomainException("Нельзя отменить доставленный заказ");
```

```
        Status = "Cancelled";
    }

    private void RecalculateTotal()
    {
        var subtotal = _items.Sum(i => i.Price * i.Quantity);
        Discount     = subtotal > 10_000m ? subtotal * 0.10m : 0m;
        TotalAmount  = subtotal - Discount;
    }
}
```

Ряд изменений носит принципиальный характер. Список `_items` приватный — добавить позицию можно исключительно через `AddItem`, который проверяет статус, лимит и выполняет пересчёт суммы. Свойства объявлены с `private set` — запись `order.Status = "Submitted"` снаружи невозможна, доступно только `order.Submit()`. Метод `Order.Create()` — создать заказ без имени клиента нельзя, поскольку некорректный объект не может быть создан в принципе.

Это и есть **инкапсуляция инвариантов**: правила, обязанные выполняться всегда, защищены самим объектом, а не расчётом на то, что весь внешний код будет их соблюдать.

Value Object

Не все объекты в предметной области обладают идентичностью. Денежная сумма, адрес, адрес электронной почты — это **Value Objects**: объекты, определяемые своими значениями, а не идентификатором. Два экземпляра `Money(100, "RUB")` полностью взаимозаменяемы.

Ключевые характеристики Value Object: **неизменяемость** и **равенство по значению**. В C# это выражается через `record`:

```
public record Money(decimal Amount, string Currency)
{
    // Валидация при создании — некорректный объект не может существовать
    public Money
    {
        if (Amount < 0)
            throw new DomainException("Сумма не может быть отрицательной");

        if (string.IsNullOrWhiteSpace(Currency))
            throw new DomainException("Валюта обязательна");

        Currency = Currency.ToUpperInvariant();
    }

    public Money Add(Money other)
```

```
    {
        if (Currency != other.Currency)
            throw new DomainException($"Нельзя сложить {Currency} и
{other.Currency}");

        return this with { Amount = Amount + other.Amount };
    }

    public Money ApplyDiscount(decimal percent)
    {
        if (percent is < 0 or > 100)
            throw new DomainException("Скидка должна быть от 0 до 100%");

        return this with { Amount = Amount * (1 - percent / 100) };
    }

    public override string ToString() => $"{Amount:N2} {Currency}";
}
```

Рассмотрим практическое последствие данного подхода:

```
// Без Value Object: string принимает любое значение, корректность не
гарантирована
public void SendConfirmation(string email) { }

// С Value Object: объект Email гарантированно прошёл валидацию при создании
public void SendConfirmation(Email email) { }
```

Во втором случае проверка корректности адреса электронной почты внутри метода излишня: если объект `Email` существует, он уже соответствует требованиям.

Aggregate и Aggregate Root

В предметных областях с нетривиальными правилами объекты не существуют изолированно — они образуют группы, изменяемые как единое целое. В DDD такая группа называется **агрегатом** (*Aggregate*), а объект, через который осуществляется любое взаимодействие с группой, — **корнем агрегата** (*Aggregate Root*).

В рассматриваемом примере класс `Order` является корнем агрегата. `OrderItem` входит в состав агрегата, но не является самостоятельной сущностью — он не существует отдельно от заказа, и обращение к нему напрямую из внешнего кода не допускается.

Это и объясняет, почему `_items` объявлен приватным, а `Items` открывает лишь `IReadOnlyList<OrderItem>`:

```
private readonly List<OrderItem> _items = new();
public IReadOnlyList<OrderItem> Items => _items.AsReadOnly();
```

Данное решение — не просто инкапсуляция в узком смысле. Это явное выражение границы агрегата: внешний код не вправе изменять коллекцию позиций в обход корня. Все изменения производятся исключительно через методы `Order`, которые гарантируют целостность состояния всего агрегата.

Важное практическое следствие: репозиторий работает с агрегатами целиком. `IOrderRepository` сохраняет и загружает `Order` со всеми его `OrderItem` — не существует отдельного `IOrderItemRepository`, поскольку позиции не имеют смысла вне контекста заказа.

Repository как доменный паттерн

В разделе 2 интерфейс `IOrderRepository` был объявлен в слое Domain — и это принципиальное решение, а не техническая деталь.

Если бы интерфейс находился в Infrastructure или Application, Domain зависел бы от других слоёв, что нарушает правило направления зависимостей. Располагая `IOrderRepository` в Domain, мы формулируем потребность предметной области: «домен нуждается в возможности сохранять и получать заказы». Способ реализации этой потребности — дело Infrastructure.

Именно поэтому методы интерфейса именуются на языке предметной области, а не в технических терминах:

```
// Язык предметной области — понятен без технических пояснений
Task<List<Order>> GetPendingAsync();

// Технический язык — требует знания о внутреннем представлении статуса
Task<List<Order>> GetByStatus(string status);
```

Сценарии применения

DDD оправдан при сложной предметной области: множество бизнес-правил, нетривиальные переходы состояний, правила, изменяющиеся независимо от технологий. DDD избыточен для простых CRUD-приложений: форма → база данных → отображение. При отсутствии бизнес-правил, требующих защиты, построение полноценной доменной модели представляет собой накладные расходы без соответствующей выгоды.

3. Dependency Injection: управление зависимостями

В предыдущих разделах приложение было разделено на слои, написаны доменные модели с поведением и обработчики сценариев использования. Возникает практический вопрос: как все эти классы соединяются друг с другом?

`CreateOrderHandler` зависит от `IOrderRepository` и `IHttpService` — кто создаёт эти объекты и передаёт их в обработчик? `OrdersViewModel` зависит от `CreateOrderHandler` — кто создаёт обработчик и передаёт его во ViewModel?

Именно этим занимается Dependency Injection.

Проблема самостоятельного создания зависимостей

При отсутствии явной передачи зависимостей класс вынужден создавать их самостоятельно:

```
public class CreateOrderHandler
{
    private readonly IOrderRepository _repository;
    private readonly IEmailService    _emailService;

    public CreateOrderHandler()
    {
        // Конкретные реализации жёстко встроены
        _repository   = new EfOrderRepository(
                            new AppDbContext(
                                new DbContextOptionsBuilder<AppDbContext>()
                                    .UseSqlite("Data Source=orders.db")
                                    .Options));
        _httpService = new HttpService("https://example.com");
    }
}
```

Это порождает три конкретные проблемы.

Невозможность подмены реализации. Для написания теста `CreateOrderHandler` требует реальный `EfOrderRepository`, которому необходима реальная база данных. Тестирование логики обработчика в изоляции от инфраструктуры становится недостижимым.

Рассредоточенность конфигурации. Строка подключения к базе данных и адрес HTTP-сервера задаются непосредственно в коде обработчика. При наличии десяти подобных классов изменение настроек потребует правок в десяти местах.

Избыточный объём знаний класса. `CreateOrderHandler` должен знать о создании заказов. Вместо этого он также знает о способе создания `AppDbContext`, хосте и порте

HTTP-сервера — это чужая область ответственности.

Принцип инверсии зависимостей

Dependency Injection реализует один из принципов SOLID — **Dependency Inversion Principle**:

> Модули верхнего уровня не должны зависеть от модулей нижнего уровня. Оба должны зависеть от абстракций.

«Верхний уровень» — бизнес-логика: `CreateOrderHandler`, `OrdersViewModel`. «Нижний уровень» — технические детали: `EfOrderRepository`, `HttpService`. Принцип предписывает: бизнес-логика не должна знать о конкретных технических реализациях. Обе стороны должны зависеть только от интерфейса между ними.

```
// Класс описывает, что ему требуется, а не способ получения этого
public class CreateOrderHandler
{
    private readonly IOrderRepository _repository;
    private readonly IHttpService    _httpService;

    public CreateOrderHandler(
        IOrderRepository repository,
        IHttpService    httpService)
    {
        _repository   = repository;
        _httpService = httpService;
    }

    // ... HandleAsync из раздела 4
}
```

`CreateOrderHandler` более не осведомлён ни о Entity Framework Core, ни о HTTP. В тесте передаётся заглушка, в рабочей среде — реальная реализация. Сам обработчик этого не знает и знать не должен.

Передача зависимостей через конструктор

Передача зависимостей через конструктор — основной и рекомендуемый способ DI. Его преимущества перед альтернативами определённы.

Зависимости явны. Изучение конструктора класса немедленно раскрывает всё необходимое для его работы — поиск скрытых зависимостей по всему коду не требуется.

Объект не может существовать без зависимостей. Передача `null` вызовет исключение при создании, а не в процессе работы. Это принцип «fail fast»: ошибка обнаруживается как можно раньше.

Конструктор как документация. Список параметров конструктора — это список областей ответственности класса. Конструктор с шестью параметрами является сигналом того, что класс выполняет слишком много функций.

```
// Конструктор явно сообщает: "данному классу требуются следующие четыре
зависимости"
public class OrdersViewModel : ViewModelBase
{
    public OrdersViewModel(
        GetOrdersHandler   getOrdersHandler,   // загрузка списка
        CreateOrderHandler createOrderHandler, // создание заказа
        CancelOrderHandler cancelOrderHandler, // отмена заказа
        IDialogService     dialogService)      // диалоги подтверждения
    {
        // ...
    }
}
```

Тестирование без инфраструктуры

Передача зависимостей через конструктор делает тестирование прямолинейным. Вместо реальных зависимостей передаются заглушки:

```
// Тест без базы данных, без HTTP, без оконного интерфейса
var fakeRepository = new FakeOrderRepository();
var fakeHttp       = new FakeHttpService();
var handler         = new CreateOrderHandler(fakeRepository, fakeEmail);

var command = new CreateOrderCommand(
    CustomerName:  "Иванов",
    CustomerHttp: "https://example.com",
    Items: new List<OrderItemDto> { new("Книга", 1, 500m) });

var orderId = await handler.HandleAsync(command);

Assert.NotEqual(Guid.Empty, orderId);
Assert.Single(fakeRepository.Orders);
Assert.Single(fakeHttp.SentMessages);
```

Данный результат является прямым следствием совместной работы трёх концепций: слоистая архитектура изолировала логику от инфраструктуры, доменная модель сосредоточила бизнес-правила в одном месте, DI сделал зависимости явными и заменяемыми.

DI-контейнер

Передача зависимостей через конструктор устраняет связанность, однако порождает новую задачу: кто-то должен создать все объекты и передать их друг другу. При ручной сборке это быстро становится неуправляемым:

```
// Ручная сборка — и это ещё небольшой пример
var dbContext      = new AppDbContext(dbOptions);
var repository     = new EfOrderRepository(dbContext);
var httpService   = new HttpService("https://example.com");
var createHandler  = new CreateOrderHandler(repository, httpService);
var cancelHandler  = new CancelOrderHandler(repository, httpService);
var getHandler     = new GetOrdersHandler(repository);
var dialogService  = new DialogService();
var ordersVm       = new OrdersViewModel(getHandler, createHandler,
cancelHandler, dialogService);
```

При пятидесяти классах объём кода инициализации становится неприемлемым. Добавление нового параметра в любой конструктор требует правок во всём этом коде.

DI-контейнер автоматизирует сборку. Вместо ручного создания регистрируются правила — «при запросе `IOrderRepository` создавать `EfOrderRepository`» — и контейнер самостоятельно разрешает граф зависимостей для построения требуемого объекта.

Стандартный DI-контейнер в .NET — `Microsoft.Extensions.DependencyInjection`:

```
dotnet add package Microsoft.Extensions.DependencyInjection
```

Мы создаем контейнер через `ServiceCollection`, после чего добавляем в него правила создания объектов и их зависимости.

Этот процесс называется **регистрацией сервисов**. В примере используются три способа: сопоставление интерфейса с реализацией, регистрация классов напрямую и использование специализированных методов расширения (как для `DbContext`).

После завершения конфигурации мы вызываем `BuildServiceProvider()`, превращая «список правил» в готовый к работе **DI-контейнер**. Теперь при запросе любого сервиса (например, `CreateOrderHandler`), контейнер автоматически создаст его и внедрит в него все необходимые зависимости, соблюдая заданное время жизни.

```
using Microsoft.Extensions.DependencyInjection;

var services = new ServiceCollection();

// При запросе IOrderRepository создавать EfOrderRepository
```

```
services.AddTransient<IOrderRepository, EfOrderRepository>();
services.AddTransient<IHttpService, HttpService>();

// Конкретные классы без интерфейса
services.AddTransient<CreateOrderHandler>();
services.AddTransient<GetOrdersHandler>();

// DbContext регистрируется через специальный метод с параметрами настройки
services.AddDbContext<AppDbContext>(options =>
    options.UseSqlite("Data Source=orders.db"));

// Построение провайдера — после этого регистрация недоступна
var provider = services.BuildServiceProvider();

// Запрос объекта — контейнер самостоятельно строит всю цепочку
зависимостей:
// AppDbContext → EfOrderRepository → HttpService → CreateOrderHandler
var handler = provider.GetRequiredService<CreateOrderHandler>();
```

`GetRequiredService<T>` — в отличие от `GetService<T>` — генерирует исключение при отсутствии зарегистрированного сервиса. Использование этого метода для обязательных зависимостей позволяет обнаружить ошибку конфигурации при старте приложения, а не в момент первого обращения к незарегистрированному сервису.

Жизненные циклы: Transient, Scoped, Singleton

Ключевым параметром при регистрации является **время жизни объекта**.

Transient — новый экземпляр при каждом запросе:

```
services.AddTransient<CreateOrderHandler>();
```

При каждом обращении к контейнеру создаётся новый экземпляр. Применяется для объектов без состояния (*stateless*). Большинство обработчиков и сервисов регистрируются как Transient.

Singleton — единственный экземпляр на протяжении всего жизненного цикла приложения:

```
services.AddSingleton<INavigationService, NavigationService>();
```

Объект создаётся однократно и существует до завершения работы приложения. Применяется для сервисов, хранящих глобальное состояние или требующих значительных затрат на создание: навигация, кэш, конфигурация. Singleton обязан быть потокобезопасным, поскольку к нему могут обращаться из нескольких потоков одновременно.

Scoped — единственный экземпляр в рамках явно ограниченной области:

```
services.AddScoped<AppDbContext>();
```

Например, в desktop-приложении scope создаётся вручную для операций, требующих общего `DbContext`. Например, при сохранении заказа с позициями необходимо, чтобы основной и вспомогательный обработчики использовали одну транзакцию:

```
using var scope = _provider.CreateScope();
var context = scope.ServiceProvider.GetRequiredService<AppDbContext>();

// Все объекты, запрошенные в данном scope, получат один и тот же context
var orderHandler =
scope.ServiceProvider.GetRequiredService<CreateOrderHandler>();
var historyHandler =
scope.ServiceProvider.GetRequiredService<CreateHistoryHandler>();

// Оба работают в едином DbContext — изменения попадут в одну транзакцию
await orderHandler.HandleAsync(command);
await historyHandler.RecordAsync(orderId, "Создан");
await context.SaveChangesAsync();
```

Практическое правило выбора жизненного цикла:

```
Нет состояния, создание недорогое      → Transient
Глобальное состояние, создание дорогое → Singleton
Требуется общая единица работы         → Scoped
```

Composition Root

Все регистрации концентрируются в единственном месте — **Composition Root**. Это единственное место в приложении, где известны конкретные реализации: что за `IOrderRepository` стоит `EfOrderRepository`, что за `IHttpService` — `HttpService`. Весь остальной код работает исключительно с интерфейсами.

В WPF-приложении Composition Root располагается в `App.xaml.cs`:

```
public partial class App : Application
{
    private IServiceProvider? _services;

    protected override void OnStartup(StartupEventArgs e)
    {
        base.OnStartup(e);
        _services = ConfigureServices();
```

```
        // Миграция базы данных при запуске
        using var scope = _services.CreateScope();
        var context = scope.ServiceProvider.GetRequiredService<AppDbContext>
();
        context.Database.Migrate();

        var mainWindow = new MainWindow
        {
            DataContext = _services.GetRequiredService<MainViewModel>()
        };
        mainWindow.Show();
    }

    private static IServiceProvider ConfigureServices()
    {
        var services = new ServiceCollection();

        // ── Infrastructure
──────────────────────────────────────────────────────
        services.AddDbContext<AppDbContext>(options =>
            options.UseSqlite("Data Source=orders.db"));

        services.AddTransient<IOrderRepository, EfOrderRepository>();
        services.AddTransient<IHttpService, HttpService>();

        // ── Application
──────────────────────────────────────────────────────────
        services.AddTransient<CreateOrderHandler>();
        services.AddTransient<CancelOrderHandler>();
        services.AddTransient<GetOrdersHandler>();

        // ── Presentation
────────────────────────────────────────────────────────────
        services.AddSingleton<IDialogService, DialogService>();
        services.AddSingleton<INavigationService, NavigationService>();
        services.AddTransient<MainViewModel>();
        services.AddTransient<OrdersViewModel>();
        services.AddTransient<CreateOrderViewModel>();

        return services.BuildServiceProvider();
    }
}
```

`MainWindow` создаётся вручную, а его `DataContext` запрашивается у контейнера. Контейнер самостоятельно создаёт `MainViewModel` со всеми его зависимостями — без единой строки ручной сборки.

Замена реализации — одна строка

Преимущество DI наглядно проявляется при необходимости замены реализации. Для разработки без реальной базы данных достаточно одной правки в Composition Root:

```
// Рабочая среда — реальная база данных
services.AddTransient<IOrderRepository, EfOrderRepository>();

// Разработка без базы данных
services.AddTransient<IOrderRepository, FakeOrderRepository>();
```

Всё остальное остаётся без изменений.